

Primitive Recursion on Algebraic Datatypes

Lecture Notes Transcription

Course: Interactive Theorem Proving (7CCSACTP)

1. Context & Binary Tree Datatype Definition

Previously Covered:

- Isabelle interface & basic reasoning (FW, BW, subst, induction on nats)
- Lists (recursive functions, structural induction)
- Algebraic datatypes (nat, list, tree)

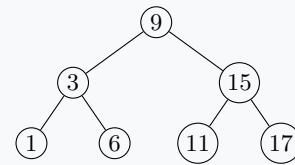
This Video:

- Primitive recursion on user-defined algebraic datatypes

Datatype Definition in Isabelle:

```
datatype 'a tree = Leaf | Node "'a tree" 'a
                  "'a tree"
```

Example Binary Search Tree (BST):



2. Primitive Recursive Functions: tree_set

Set of Elements in a Tree:

```
tree_set Leaf = {}
tree_set (Node l a r) = insert a (tree_set l ∪ tree_set r)
```

Evaluation Trace on Example Tree:

```
tree_set T = insert 9 (tree_set T3 ∪ tree_set T15)
             = insert 9 (insert 3 (tree_set T1 ∪ tree_set T6) ∪ insert 15 (tree_set T11 ∪ tree_set T17))
             = {1, 3, 6, 9, 11, 15, 17}
```

3. Inorder Traversal of Trees

Definition of Inorder Traversal:

```
inorder Leaf = []
inorder (Node l a r) = (inorder l) @ [a] @ (inorder r)
```

BST Property: If the tree is a Binary Search Tree (where for every node, all left subtree elements are < node and all right subtree elements are > node), then **inorder** yields a **sorted list**:

[1, 3, 6, 9, 11, 15, 17]

Step-by-step Evaluation Trace:

```
inorder T9 = (inorder T3) @ [9] @ (inorder T15)
             = ((inorder T1) @ [3] @ (inorder T6)) @ [9] @ (inorder T15)
             = (([] @ [1] @ []) @ [3] @ ([] @ [6] @ [])) @ [9] @ ...
             = ([1] @ [3] @ [6]) @ [9] @ ([11] @ [15] @ [17])
             = [1, 3, 6, 9, 11, 15, 17] ✓
```